

HEPSİNİ BİR ARAYA GETİRMEK

[Kaynak](#)

İÇERİK

1. Web İsteklerinin Arka Planı: Her Şey Nasıl Çalışır?
2. Web Sunucuları Nasıl Çalışır? (Web Servers)

Web İsteklerinin Arka Planı: Her Şey Nasıl Çalışır?

Bir web sayfasını tarayıcıda açtığımızda arka planda gerçekleşen olaylar, siber güvenlikte **Enumeration** (Bilgi Toplama) ve **Exploitation** (İstismar) süreçlerinin temelini oluşturur. Bu süreci tam olarak anlamazsak, trafiği manipüle etmeyi veya zafiyetleri bulmayı bekleyemeyiz. İşte bir isteğin anatomisi:

1. İsimden Adrese: DNS (Domain Name System) Süreci

Bilgisayarlar birbirleriyle insanlar gibi isimler (<https://www.google.com/search?q=google.com>) üzerinden değil, **IP adresleri** üzerinden haberleşir. Bir URL yazdığımızda ilk durak her zaman DNS'tir.

- **Mantık:** Bilgisayarımız hedef sunucunun IP'sini bilmiyorsa, bu ismi bir IP adresine çözümlemek için DNS sunucularına sorar.
- **Neden Önemli?** Eğer bir saldırgan DNS kayıtlarını manipüle edebilirse (**DNS Spoofing**), trafiği kendi kontrolündeki sahte bir sunucuya yönlendirebilir.

2. İletişim Dili: HTTP Protokolü

IP adresi öğrenildikten sonra bilgisayarımız, web sunucusuyla konuşmak için **HTTP (Hypertext Transfer Protocol)** adı verilen özel bir komut setini kullanır. Bu sadece "veriyi ver" demek değil, belirli kuralları olan bir diyalogdur:

- **İstek (Request):** Tarayıcımız sunucuya "Bana şu sayfayı, şu formatta ver" der.
- **Yanıt (Response):** Sunucu bu isteği işler ve bir durum koduyla (200 OK, 404 Not Found vb.) birlikte içeriği gönderir.

3. Web Sayfasının Bileşenleri

Sunucu bize ham bir veri yığını gönderir. Tarayıcının görevi ise bu yığını görsel bir şölene dönüştürmektir. Gelen veriler genellikle şu dört ana parçadan oluşur:

- **HTML (Hypertext Markup Language):** Sayfanın iskeletidir. Başlıklar nerede, paragraflar nerede, hangi link nereye gidiyor; bunların hepsi HTML ile tanımlanır.

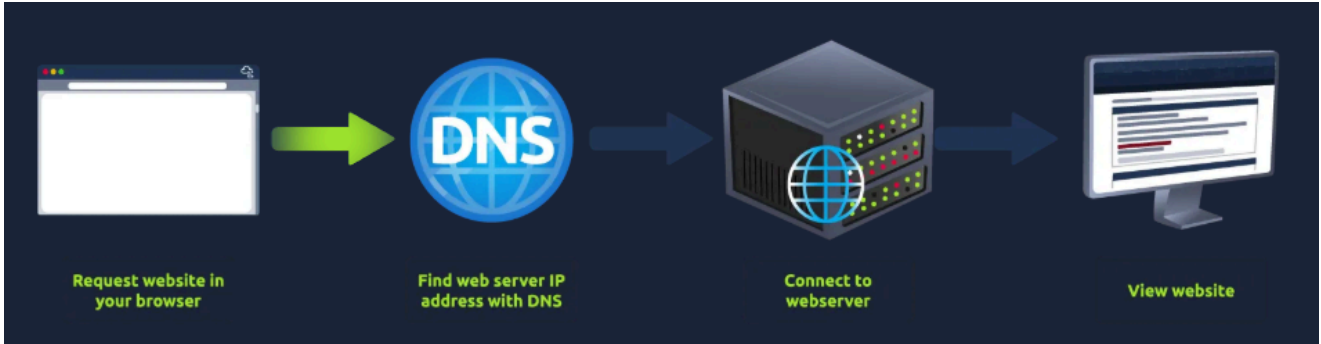
- **JavaScript (JS):** Sayfanın etkileşimli kısmıdır. Butona basınca ne olacağı, verilerin dinamik yüklenmesi gibi "hareketler" JS ile sağlanır. Sızma testlerinde **XSS (Cross-Site Scripting)** zafiyetlerinin ana odağıdır.
- **CSS (Cascading Style Sheets):** Sayfanın makyajıdır. Renkler, fontlar ve layout (düzen) CSS ile belirlenir.
- **Statik Dosyalar:** Görseller (Images), videolar ve font dosyaları gibi içeriklerdir.

Önemli Not: Tarayıcı bu dosyaların her biri için aslında ayrı birer HTTP isteği yapar. Bir web sitesini açtığınızda arka planda onlarca, bazen yüzlerce küçük istek döner.

4. Performans ve Ek Özellikler

Modern web dünyasında sadece sunucu-istemci ilişkisi yetmez. Deneyimi iyileştirmek ve güvenliği sağlamak için araya giren ek bileşenler vardır:

- **Caching (Önbelleğe Alma):** Daha önce indirilen CSS veya görsellerin tekrar tekrar istenmemesi için yerelde saklanmasıdır.
- **Load Balancers (Yük Dengeleyiciler):** Çok yoğun sitelerde trafiği birden fazla sunucuya dağıtan donanım veya yazılımlardır. Bir sızma testinde karşımızda tek bir makine mi yoksa bir sunucu çiftliği mi olduğunu anlamak için bu yapıları bilmek gerekir.
- **WAF (Web Application Firewall):** Gelen HTTP isteklerini inceleyerek zararlı olanları (örneğin bir SQL Injection denemesini) durduran güvenlik duvarıdır.



Modern Web Mimarisinin Tamamlayıcı Bileşenleri

Bir web sitesi sadece bir sunucu ve bir tarayıcıdan ibaret değildir. Ölçeklenebilirlik, hız ve güvenlik sağlamak amacıyla araya giren kritik katmanlar vardır. Bu bileşenleri anlamak, bir sızma testinde "neden engellendiğimizi" veya "neden her seferinde farklı bir sonuç aldığımızı" açıklayan anahtardır.

1. Load Balancers (Yük Dengeleyiciler)

Trafik çok arttığında veya uygulamanın **High Availability** (Yüksek Erişilebilirlik - sistemin sürekli açık kalması) sağlaması gerektiğinde tek bir sunucu yetersiz kalır. İşte burada **Load Balancer** devreye girer.

Temel Görevleri:

- **Yük Dağıtımı:** Gelen binlerce isteği arkadaki birden fazla sunucuya paylaştırarak hiçbir sunucunun çökmemesini sağlar.
- **Failover (Hata Devretme):** Eğer arkadaki sunuculardan biri bozulursa, trafiği otomatik olarak çalışan diğer sunuculara yönlendirir.

Nasıl Çalışır? Siz bir istek yaptığınızda, isteği önce Load Balancer karşılar, ardından bunu arkadaki uygun sunucuya iletir. Hangi sunucuya gideceğine şu algoritmalarla karar verir:

- **Round-Robin:** İstekleri sırayla gönderir (1. istek Sunucu A'ya, 2. istek Sunucu B'ye...).
- **Weighted (Ağırlıklı):** Sunucuların o anki yükünü kontrol eder ve en az meşgul olan veya en güçlü donanıma sahip olana isteği gönderir.

Health Check (Sağlık Kontrolü): Load Balancer, arkadaki sunucuların "hayatta" olup olmadığını anlamak için onlara periyodik olarak sinyal gönderir. Eğer bir sunucu yanıt vermezse, Load Balancer o sunucu düzelene kadar oraya trafik göndermeyi durdurur.

Pentest Notu: Bir siteye sızmaya çalışırken bazen bir payload'un çalışıp diğerinin çalışmamasının sebebi, Load Balancer'ın sizi her seferinde farklı (ve belki de farklı konfigüre edilmiş) bir arka plan sunucusuna yönlendirmesi olabilir.

2. CDN (Content Delivery Networks - İçerik Dağıtım Ağları)

CDN, yoğun trafikli sitelerin yükünü hafifletmek için harika bir çözümdür. Özellikle **statik dosyaların** (JS, CSS, görseller, videolar) dağıtımında kullanılır.

- **Mantık:** Bu dosyalar dünya geneline yayılmış binlerce sunucuda kopyalanır.
- **Fiziksel Mesafe:** Bir kullanıcı bir dosya istediğinde, CDN kullanıcının coğrafi konumuna en yakın sunucuyu tespit eder ve dosyayı oradan gönderir. Bu, verinin dünyanın öbür ucundan gelmesini engelleyerek gecikmeyi (**Latency**) minimize eder.

3. Databases (Veritabanları)

Web sunucuları kullanıcı bilgilerini, içerikleri veya yapılandırmaları saklamak için veritabanlarıyla konuşur. Veritabanları basit bir metin dosyasından, devasa sunucu kümelerine kadar farklılık gösterebilir.

Sık karşılaşacağımız popüler veritabanları:

- **İlişkisel (SQL):** MySQL, MSSQL (Microsoft), Postgres.
- **NoSQL (Döküman tabanlı):** MongoDB.

Kritik Bilgi: Web uygulaması ile veritabanı arasındaki bu iletişim düzgün temizlenmezse (sanitization yapılmazsa), karşımıza en tehlikeli zafiyetlerden biri olan **SQL Injection** çıkar.

4. WAF (Web Application Firewall - Web Uygulaması Güvenlik Duvarı)

WAF, web isteđi ile sunucu arasında duran bir "fedai" gibidir. Birinci önceliđi sunucuyu hack girişimlerinden ve **DoS (Denial of Service)** saldırılarından korumaktır.

WAF Neleri Kontrol Eder?

- **Saldırı Analizi:** İstekleri tarayarak içinde bilinen saldırı paternleri (SQLi, XSS, Path Traversal vb.) olup olmadığına bakar.
- **Bot Kontrolü:** İsteđin gerçek bir tarayıcıdan mı yoksa bir bottan mı geldiđini anlamaya çalışır.
- **Rate Limiting (Hız Sınırlandırma):** Bir IP adresinden saniyede kaç istek geldiđini takip eder. Eğer birisi (veya bir script) çok hızlı istek atıyorsa, o IP'yi geçici olarak engeller.

Sonuç: Eğer bir istek potansiyel bir saldırı olarak işaretlenirse, WAF bu isteđi çöpe atar ve sunucuya ulaşmasına asla izin vermez.

Web Sunucuları Nasıl Çalışır? (Web Servers)

Web sunucuları, internetin dijital kütüphanecileridir. Bir tarayıcıdan gelen istekleri dinler, işler ve istenen dosyaları (veya hata mesajlarını) geri gönderirler. Bir siber güvenlik uzmanı için bu mekanizmayı bilmek, dosyaların nerede saklandığını ve sunucu taraflı kodların nasıl suistimal edilebileceđini anlamak demektir.

1. Web Sunucusu Nedir?

Teknik olarak bir web sunucusu, gelen bağlantıları dinleyen ve **HTTP** protokolünü kullanarak istemcilere (bizim tarayıcımıza) içerik sunan bir yazılımdır.

En Yaygın Web Sunucu Yazılımları:

- **Apache ve Nginx:** Linux ekosisteminin devleri.
- **IIS (Internet Information Services):** Microsoft Windows tabanlı sunucuların standardı.
- **NodeJS:** JavaScript tabanlı modern, asenkron sunucu yapısı.

Root Directory (Kök Dizin) Kavramı

Web sunucusu, dosyaları işletim sistemindeki herhangi bir yerden değil, ayarlarında belirtilen belirli bir klasörden sunar. Buna **Root Directory** denir.

- **Linux (Apache/Nginx):** Genellikle `/var/www/html` dizinidir.
- **Windows (IIS):** Genellikle `C:\inetpub\wwwroot` dizinidir.

Örnek Senaryo: Sen tarayıcına `http://site.com/resim.jpg` yazdığında, sunucu aslında kendi yerel diskindeki `/var/www/html/resim.jpg` dosyasını bulur ve sana gönderir.

2. Sanal Hostlar (Virtual Hosts)

Bir fiziksel sunucunun içinde onlarca farklı web sitesi barındırılabilir. Peki, sunucu hangi isteğin hangi siteye (örneğin `site-a.com` mu yoksa `site-b.com` mu) ait olduğunu nasıl anlar? İşte burada **Virtual Hosts** devreye girer.

Mantık Zinciri:

1. Sen bir siteye istek attığında, tarayıcın HTTP başlığına (Header) **Host** bilgisini ekler.
2. Web sunucusu yazılımı, bu başlığı okur ve kendi yapılandırma dosyalarındaki (text-based config files) kayıtlarla karşılaştırır.
3. Eğer eşleşme varsa, o siteye özel olarak atanmış dizindeki dosyaları sunar.
4. Eşleşme yoksa, genellikle sistemde tanımlı olan **varsayılan (default)** web sitesini gösterir.

Dizin Eşleşme Örneği:

- `site1.com` isteği gelirse -> `/var/www/site_bir` klasörüne bak.
- `site2.com` isteği gelirse -> `/var/www/site_iki` klasörüne bak.

3. Statik vs. Dinamik İçerik

Bu ayrım, sızma testlerinde saldırı yüzeyini (attack surface) belirleyen en kritik noktadır.

Statik İçerik

Adı üzerinde; değişmeyen içeriktir. Sunucu dosyayı olduğu gibi diskten alır ve sana iletir.

- **Örnekler:** Resimler, CSS dosyaları, JavaScript dosyaları ve basit HTML sayfaları.
- **Güvenlik:** Genellikle düşük risklidir çünkü sunucu tarafında bir kod çalıştırılmaz.

Dinamik İçerik

İsteğe, kullanıcıya veya zamana göre değişen içeriktir. Örneğin bir blog sitesinin ana sayfası, her yeni yazı eklendiğinde otomatik güncellenir.

- **Backend (Arka Plan):** Tüm bu değişimler arka planda, programlama dilleri aracılığıyla yapılır.
- **Frontend (Ön Yüz):** Tarayıcıda gördüğümüz HTML sonucudur. Biz backend kodunu asla göremeyiz, sadece o kodun **ürettiği** sonucu görürüz.

4. Scripting ve Backend Dilleri

Web sitelerini etkileşimli hale getiren motorlar bunlardır. Popüler olanlar: **PHP, Python, Ruby, NodeJS, Perl**. Bu diller; veritabanlarına erişebilir, dosya sisteminde işlem yapabilir ve kullanıcıdan gelen veriyi işleyebilir.

Canlı Bir Örnek: PHP Nasıl Çalışır?

Diyelim ki bir URL'ye şu isteği attık: `http://example.com/index.php?name=not_tut`

Sunucu Tarafındaki Kod (`index.php`):

HTML

```
<html>
  <body>
    Hello <?php echo $_GET["name"]; ?>
  </body>
</html>
```

İşleyiş:

1. Sunucu bu dosyayı açar, `<?php ... ?>` etiketleri arasındaki kodu görür.
2. `$_GET["name"]` kısmına URL'den gelen `not_tut` değerini yerleştirir.
3. Kodu çalıştırır ve temiz bir HTML üretir.

İstemciye (Bize) Gelen Çıktı:

HTML

```
<html>
  <body>
    Hello not_tut
  </body>
</html>
```

Önemli Güvenlik Notu: İstemci asla PHP kodunu görmez. Ancak, eğer geliştirici kullanıcıdan gelen `name` verisini filtrelemeden ekrana basarsa, biz buraya zararlı bir kod (Payload) göndererek siteyi hackleyebiliriz. Bu etkileşim noktaları, web açıklarının (XSS, SQLi vb.) doğduğu yerdir.